MSc programme Applied Artificial Intelligence

Programming for AI: Final Project Report

Forest Cover Type Classification

Topic 37 · Cluster 7: Machine Learning on Tabular Data

Team member	Group	Main responsibility
Amanbayev Marlen	AAI-2601M	Data engineering, SQLite & SQL, application (menu, Streamlit)
Aktaev Erik	AAI-2601M	Data quality audit, exploratory analysis, visualisation
Gajiyev Rashad	AAI-2601M	Modelling, tuning, evaluation, interpretation

Repository: https://github.com/mara-werils/forest-cover-classification

November 2026

Summary

We predict which of seven forest cover types grows on a 30 × 30 m cell of the Roosevelt National Forest (Colorado, USA) from cartographic and soil variables only. The UCI Covertype data set has 581,012 cells and is strongly imbalanced: Lodgepole Pine covers 48.76% of the cells, Cottonwood/Willow only 0.47% (103.1× fewer). The data are loaded in chunks with compact data types (243.8 → 33.8 MB), stored as Parquet and in a normalised SQLite database that is queried with 12 analytical SQL queries.

On a stratified hold-out test split of 116,203 cells, gradient boosting (LightGBM, class-weighted) reaches F1-macro 0.9558 and accuracy 0.9736, against 0.0937 for the majority-class baseline and 0.5478 for multinomial Logistic Regression. Random Forest (0.9489) and HistGradientBoosting (0.9546) are close, but the Random Forest model is 6× larger. 69% of the remaining errors are confusions between Spruce/Fir and Lodgepole Pine, the two species that share the same mid-elevation terrain. A leave-one-area-out check shows the limit of this result: on a wilderness area that was not used for training, F1-macro drops to 0.24–0.35, so the model maps the surveyed areas well but does not transfer to new terrain without local training data.

1. Problem and objectives

Forest inventories are expensive to collect in the field, while terrain variables are available from digital elevation models for every cell of a map. A model that predicts the dominant tree species from terrain alone can fill gaps in inventories of remote or protected areas. The project objectives follow cluster 7 of the assignment:

- load and audit a data set of more than 500,000 rows with explicit memory control;
- store it in a normalised relational database and analyse it with SQL;
- compare a baseline with Logistic Regression, Random Forest and gradient boosting on a leakage-free split, with hyper-parameters chosen by cross-validation;
- judge the models by F1-macro (all classes weigh the same), per-class metrics, training time, prediction time, memory and model size;
- interpret the best model (permutation importance, confusion matrix, error analysis);
- provide a console menu and a Streamlit dashboard where a user enters feature values and gets a prediction.

2. Data

Source: UCI Machine Learning Repository, Covertype (Blackard & Dean, 1998; CC BY 4.0). Each row is a 30 × 30 m cell in one of four wilderness areas of the Roosevelt National Forest. The cover type was taken from US Forest Service Region 2 inventory data; the predictors were derived from US Geological Survey and USFS data.

Table 1. Structure of the data set (581,012 rows × 55 columns)

Group	Columns	Type / unit
Terrain	Elevation, Aspect, Slope	metres, degrees azimuth, degrees
Distances	Horizontal & vertical distance to water, distance to roads, to wildfire ignition points	metres
Illumination	Hillshade at 9am, noon, 3pm (summer	index 0–255

	solstice)	
Wilderness area	4 one-hot columns (Rawah, Neota, Comanche Peak, Cache la Poudre)	binary
Soil type	40 one-hot columns (USFS Ecological Landtype Units)	binary
Target	Cover_Type 1–7	class

Table 2. Target distribution

Cover type	Cells	Share, %
Spruce/Fir	211,840	36.46
Lodgepole Pine	283,301	48.76
Ponderosa Pine	35,754	6.15
Cottonwood/Willow	2,747	0.47
Aspen	9,493	1.63
Douglas-fir	17,367	2.99
Krummholz	20,510	3.53

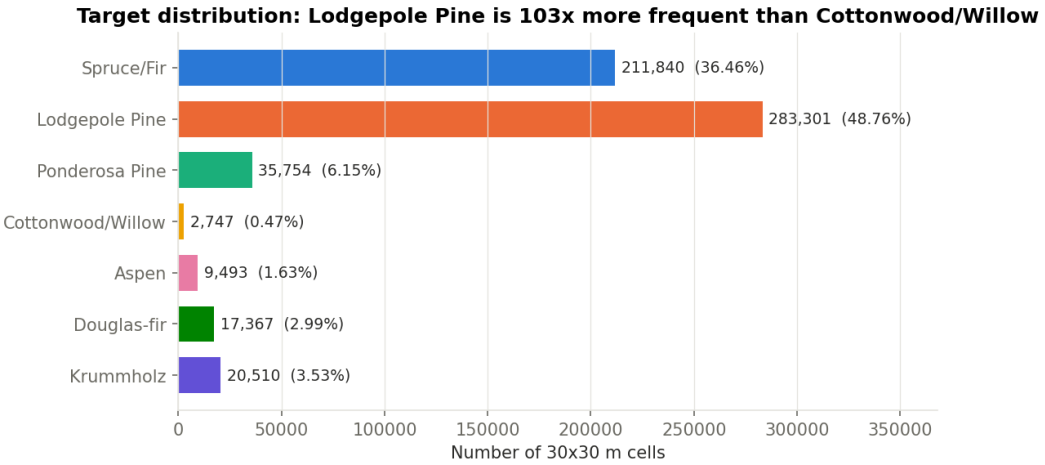


Figure 1. Class imbalance: the two majority classes hold 85% of the cells.

3. Data processing

3.1 Memory-aware loading and Parquet

The archive is downloaded by the code and unpacked to a CSV with a header. A default pd.read_csv stores all 55 columns as int64. We read the file in chunks of 100,000 rows with the smallest lossless dtype per column: int16 for elevation and distances (the vertical distance is signed), uint16 for aspect, uint8 for slope, hillshade, the one-hot flags and the target. The 4 + 40 one-hot columns are then collapsed into two categorical columns (Wilderness 1–4, Soil 1–40), and the frame is written to Parquet (snappy).

Table 3. Memory before and after dtype optimisation (−86.1%)

Method	DataFrame, MB	Time, s
Default read (int64)	243.8	0.8
Chunked read, compact dtypes	33.8	0.69

Table 4. CSV vs Parquet (median of repeated loads; peak RAM traced by tracemalloc)

Format	File, MB	Load time, s	Peak RAM, MB	Frame, MB
CSV, default dtypes	71.69	0.897	487.8	243.8
CSV, compact dtypes	71.69	0.808	127.7	33.8
Parquet, all columns	6.91	0.083	7.3	34.9
Parquet, 3 columns	6.91	0.007	1.1	2.2

Parquet is 10× smaller on disk and loads 10.8× faster than the CSV, and because it is columnar a query that needs three columns reads only those (7 ms). Parquet is therefore the working format of every later step.

3.2 Data quality audit

Table 5. Audit results and cleaning decisions

Check	Result	Decision
Missing values	none	imputers kept in pipelines only for user input
Exact duplicate rows	0	nothing to remove
Same features, different label	0	no irreducible label noise
Rows with 0 or > 1 soil flag	0	one-hot blocks are valid → safe to collapse
Values outside physical range	0	none
IQR outliers (max per column)	5.42% (vertical distance to water)	kept: real terrain, not errors
Hillshade_3pm = 0	1338	kept: full afternoon shadow is plausible
Negative vertical distance	55,143	kept: cell lies below the nearest water

The data are clean, so no rows were removed. Outliers flagged by the 1.5 × IQR rule are physically meaningful extremes (very remote cells, steep slopes); removing them would delete exactly the cells of rare classes such as Krummholz at high elevation. Tree ensembles are insensitive to monotone scale, and for the linear model the features are standardised inside the pipeline.

4. Exploratory analysis

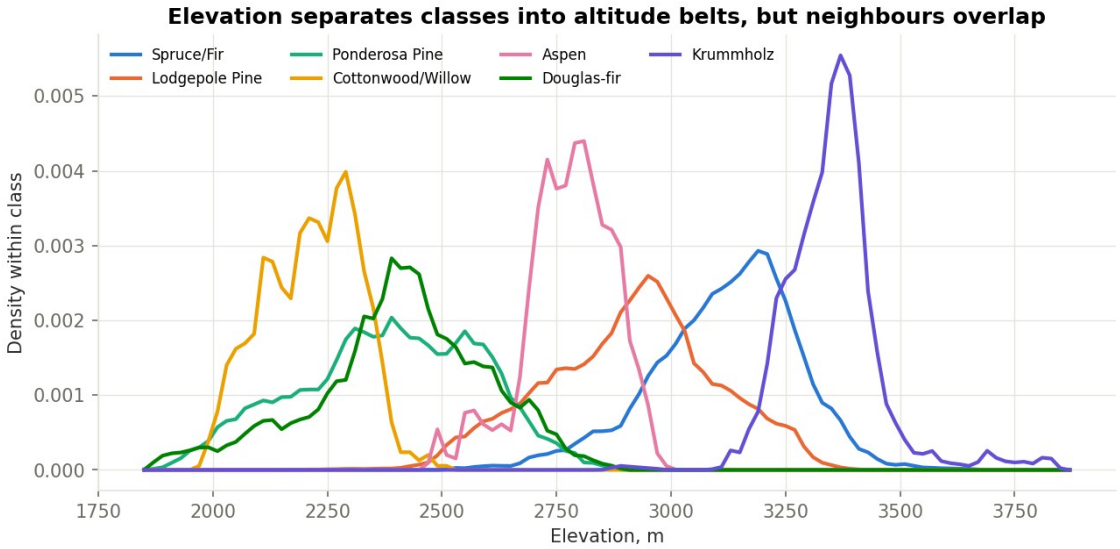


Figure 2. Elevation density by class: each species occupies an altitude belt.

Elevation is the dominant signal: Cottonwood/Willow, Ponderosa Pine and Douglas-fir live below ≈2,600 m, Lodgepole Pine and Aspen in the middle, Spruce/Fir higher and Krummholz (wind-shaped trees at the tree line) above ≈3,300 m. The belts overlap, which is why elevation alone is not enough and why the Spruce/Fir – Lodgepole Pine pair is hard.

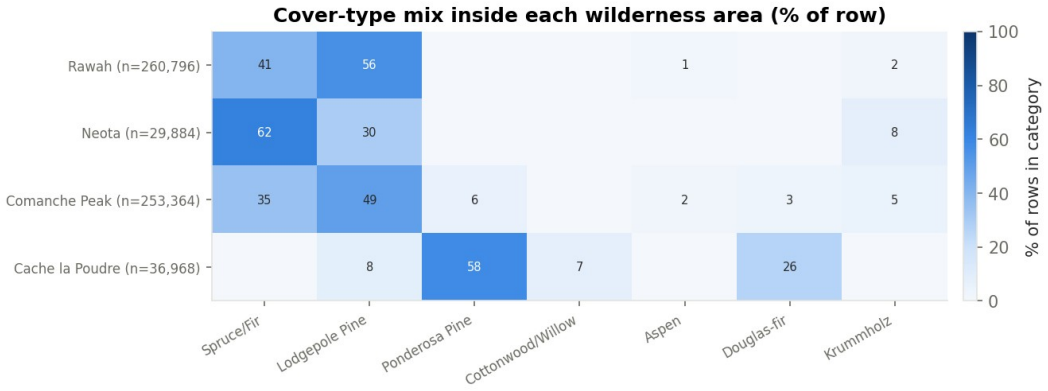


Figure 3. Cover-type mix inside each wilderness area (row %).

Wilderness area acts like a strong prior: Cache la Poudre is the only area with Cottonwood/Willow and holds most Ponderosa Pine and Douglas-fir, while Neota is dominated by Spruce/Fir. Soil types are even more specific (see Q03 below).

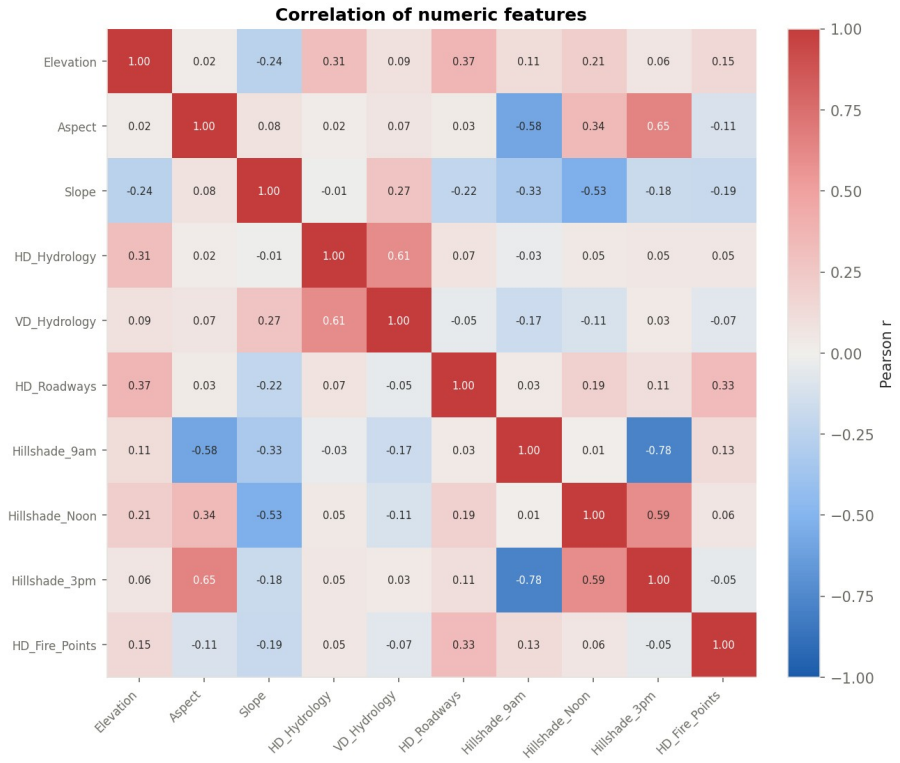


Figure 4. Pearson correlations of the numeric features.

The strongest correlations are among the illumination variables (Hillshade_9am ~ Hillshade_3pm r = −0.78; aspect ~ hillshade r ≈ ±0.6), which are all functions of slope orientation, and between the horizontal and vertical distance to water (r = 0.61). No pair is collinear enough to justify dropping a feature for trees; the linear model uses L2 regularisation. Boxplots by class (reports/figures/03_boxplots_by_class.png) and an interactive Plotly scatter (09_interactive_scatter.html) complete the EDA.

5. Database and SQL

The cleaned data are loaded into SQLite with a normalised schema (Figure 5). The 44 one-hot columns become two foreign keys of the fact table cell; soil types are linked to climatic and geologic zones decoded from the first two digits of their USFS ELU code. Experiments, models, the train/test assignment and the test-set predictions are stored in the same database so that model errors can be analysed with SQL. The fact table is inserted in chunks of 100,000 rows (to_sql with chunksize).

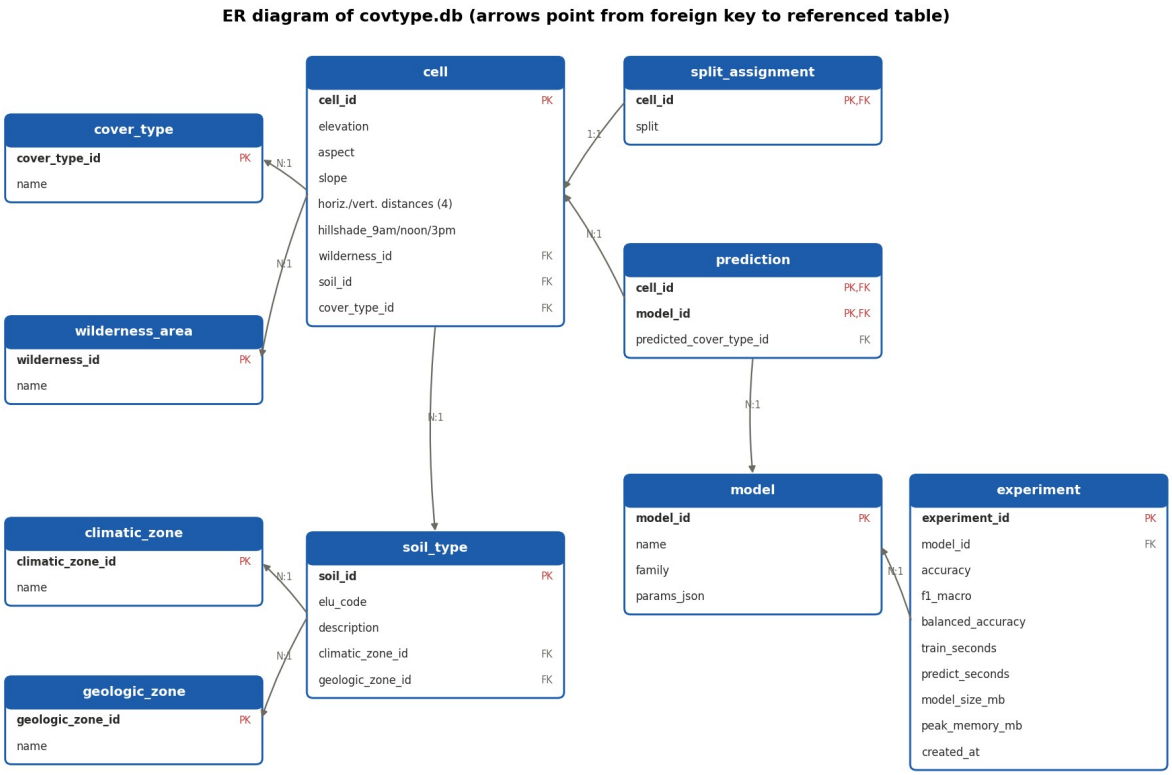


Figure 5. ER diagram of covtype.db.

Table 6. The 12 analytical queries (src/queries.py)

Query	SQL features	Rows
Q01 Class distribution with share of total	JOIN, GROUP BY, window SUM() OVER ()	7
Q02 Average terrain per cover type and wilderness area	2 JOINs, GROUP BY, HAVING	17
Q03 Dominant cover type of every frequent soil type	CTE, JOIN, GROUP BY/HAVING, ROW_NUMBER() OVER (PARTITION BY)	31
Q04 Cover-type mix by 200 m elevation band	CTE, CASE, GROUP BY, window share within band	41
Q05 Cover types per climatic zone of the soil	3 JOINs through soil_type, GROUP BY	34
Q06 Classes whose mean elevation is above the overall mean, and spread	Subquery in FROM, scalar subquery in WHERE, JOIN, GROUP BY	2
Q07 Rank of wilderness areas by distance to roads within each class	CTE, JOIN, RANK() OVER (PARTITION BY … ORDER BY)	17
Q08 Class mix in elevation quartiles inside each wilderness area	Subquery in FROM, NTILE() OVER (PARTITION BY), GROUP BY	16
Q09 Rare soil types (< 1000 cells) and the classes they host	JOIN, GROUP BY, HAVING, GROUP_CONCAT	9
Q10 Per-class recall of every model on the test split	4 JOINs, CTE, GROUP BY, conditional aggregation	42
Q11 Top-2 confusions per true class for the	CTE, JOINs, ROW_NUMBER() OVER	14

best model	(PARTITION BY), subquery	
Q12 Error rate of the best model by wilderness area and elevation band	CTE, JOINs, GROUP BY/HAVING, LAG() OVER	16

Selected findings. Q03 (CTE + ROW_NUMBER) shows that soil is highly informative: in soil 12 (ELU 4744, 29,971 cells) 91.0% of cells are Lodgepole Pine. Q08 (NTILE within each area) shows the altitude succession inside Comanche Peak: Spruce/Fir grows from 7.4% of the lowest elevation quartile to 59.6% of the highest, where Krummholz appears (20.5%). Q10–Q12 join predictions with cells to compute per-class recall of every model, the most frequent confusions and the error rate per area and elevation band (LAG compares adjacent bands).

Every query result is loaded into pandas with pd.read_sql_query and saved to reports/sql/. As a check, the same aggregation was run in pandas and in SQLite: results are identical (17 groups); pandas on Parquet took 0.088 s including the read, SQLite 0.432 s. For a single-user analytical workload that fits in memory, pandas + Parquet is faster; SQL gives a persistent, typed, joinable store that other tools (and the Streamlit app) can query without loading everything.

6. Methods

6.1 Split and leakage control

The data have no time stamp or group identifier, so we use a stratified random hold-out split: 80% train (464,809 cells), 20% test (116,203 cells), seed 42. The split is made before any fitting. All preprocessing is inside a scikit-learn Pipeline/ColumnTransformer and is fitted on the training part only; hyper-parameters are chosen by cross-validation inside the training part. The test split is used once, for the final comparison.

6.2 Features and preprocessing

- **Engineered features (stateless, row-wise):** Euclidean distance to water, water elevation (elevation − vertical distance), mean hillshade, hydrology + fire distance, |hydrology − road| and |fire − road| distances, sin/cos of aspect (an angle: 359° ≈ 1°).
- **Logistic Regression:** median imputation and standard scaling of 18 numeric columns, one-hot encoding of wilderness and soil (44 columns).
- **Tree models:** no scaling; wilderness and soil kept as integer codes (HistGradientBoosting treats them as native categorical features).

6.3 Models and tuning

Five approaches are compared: DummyClassifier (most frequent class, the baseline), multinomial Logistic Regression, Random Forest, scikit-learn HistGradientBoosting and LightGBM. Each model is tuned with RandomizedSearchCV (3-fold stratified CV, scoring = F1-macro) on a stratified 40,000-row subsample of the training split; the subsample keeps the search within ≈1 hour on a laptop CPU. The best configuration is refitted on the full training split. Class imbalance is handled by searching class_weight ∈ {None, balanced}; for LightGBM the opposite weighting was also trained as a controlled check.

Table 7. Hyper-parameter search (40k-row subsample, 3-fold CV)

Model	Searched	Chosen	CV F1-macro
Logistic Regr.	C, class_weight	class_weight=None, C=10.0	0.5666

Random Forest	class_weight, max_features, min_samples_leaf, n_estimators	n_estimators=100, min_samples_leaf=1, max_features=sqrt, class_weight=None	0.8008
HistGB	class_weight, l2_regularization, learning_rate, max_iter, max_leaf_nodes	max_leaf_nodes=255, max_iter=300, learning_rate=0.05, l2_regularization=0.0, class_weight=balanced	0.8455
LightGBM	class_weight, learning_rate, min_child_samples, n_estimators, num_leaves	num_leaves=255, n_estimators=500, min_child_samples=10, learning_rate=0.05, class_weight=balanced	0.847

6.4 Metrics and resource measurement

Quality: accuracy, F1-macro (main metric, every class weighs the same), weighted F1, balanced accuracy and per-class precision/recall/F1. Cost: wall-clock training time on the full training split, prediction time per 1,000 rows, peak additional resident memory during training (sampled every 50 ms with psutil, which also sees LightGBM's C++ allocations), and the size of the pickled pipeline. Timings were measured on a MacBook Air (Apple Silicon, 16 GB) with 4 threads while other jobs were running, so they are comparable with each other but not absolute.

7. Results

Table 8. Test-split results (116,203 cells). Best F1-macro: LightGBM (balanced)

Model	Accuracy	F1-macro	Bal. acc.	Train, s	Predict, ms/1k	Peak RAM, MB	Size, MB
Baseline (Dummy)	0.4876	0.0937	0.1429	1.4	0.4	190	0.00
Logistic Regr.	0.7308	0.5478	0.5245	43.2	1.0	634	0.01
Random Forest	0.9718	0.9489	0.9390	62.3	10.3	572	568.02
HistGB	0.9698	0.9546	0.9612	327.6	100.7	450	59.75
LightGBM (balanced)	0.9736	0.9558	0.9590	171.8	401.6	471	90.96
LightGBM (no weights)	0.9742	0.9537	0.9472	182.5	311.8	323	89.12

Table 9. Threshold-free metrics on the test split (predicted probabilities)

Model	ROC-AUC (OvR, macro)	PR-AUC (macro)	F1-macro
Baseline (Dummy)	0.5000	0.1429	0.0937
Logistic Regr.	0.9383	0.6252	0.5478
Random Forest	0.9992	0.9881	0.9489

HistGB	0.9989	0.9897	0.9546
LightGBM (balanced)	0.9991	0.9903	0.9558
LightGBM (no weights)	0.9991	0.9896	0.9537

ROC-AUC is close to 1 for every tree ensemble, so it hardly separates them; the macro PR-AUC, which is driven by the rare classes, is more informative and gives the same ranking at the top (LightGBM 0.9903, HistGB 0.9897, Random Forest 0.9881). Logistic Regression reaches a respectable ROC-AUC (0.9383) but a PR-AUC of only 0.6252: its probabilities rank cells reasonably overall but fail on the rare classes.

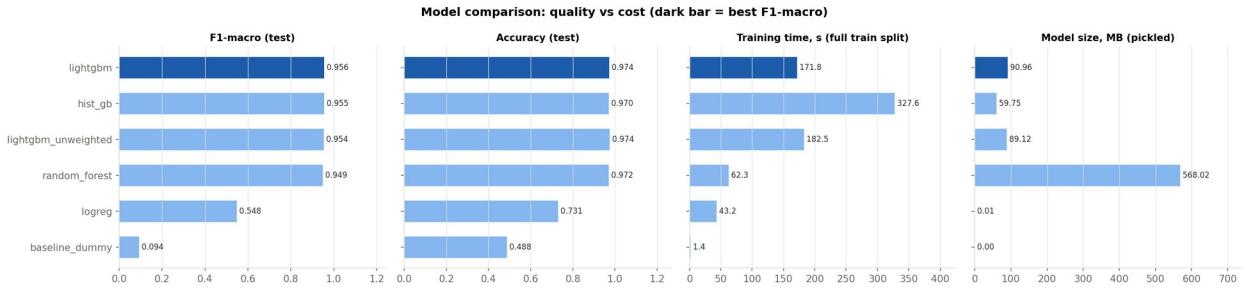


Figure 6. Quality vs cost of the compared models.

All tree ensembles are far above the baseline and the linear model. LightGBM (balanced) has the best F1-macro (0.9558); its accuracy (0.9736) is within 0.001 of the best one (unweighted LightGBM, 0.9742). HistGradientBoosting is within 1.2 thousandths in F1-macro, has a slightly higher balanced accuracy (0.9612 vs 0.9590) and a smaller model, but trains about twice as long and has lower accuracy (0.9698). Random Forest trains fastest among the strong models (62 s) and predicts 39× faster, but its fully grown trees take 568 MB.

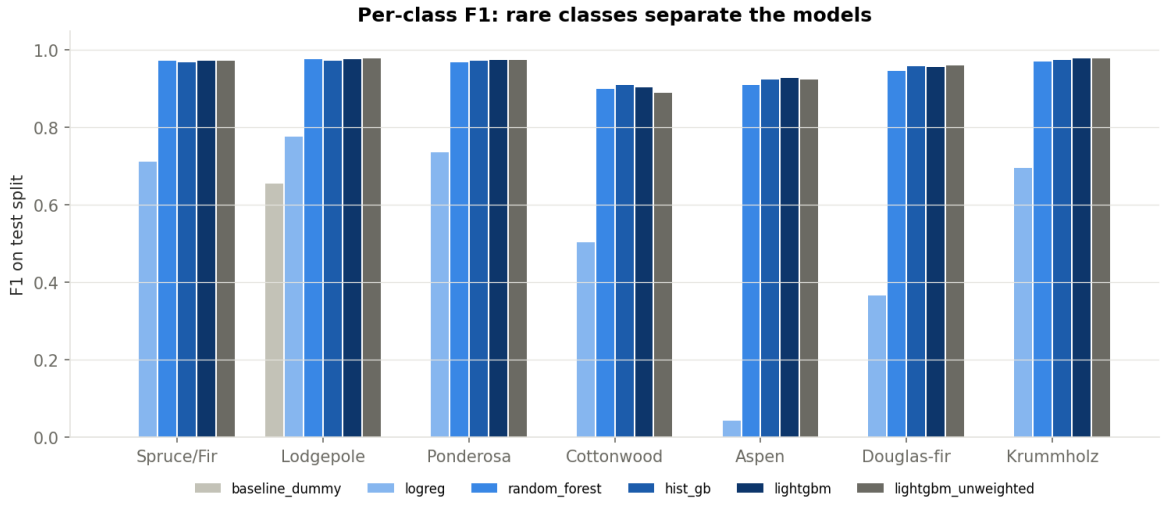


Figure 7. Per-class F1 of every model.

The per-class view explains the gap in F1-macro. Logistic Regression is reasonable on the large classes but fails on Aspen (F1 0.043) and Douglas-fir (0.366): these classes are defined by combinations (an elevation belt within a given area and soil), which a linear decision function in the original features cannot express. Tree ensembles learn such interactions directly. The hardest class for every strong model is Cottonwood/Willow (0.47% of data).

7.1 Class imbalance

Class weighting changes LightGBM only slightly but in the expected direction: balanced weights raise balanced accuracy from 0.9472 to 0.9590 and F1-macro from 0.9537 to 0.9558, while accuracy stays at 0.974. Cottonwood/Willow F1 improves from 0.889 to 0.904. The effect is small because with 465k training rows even the rarest class has ≈2,200 examples.

A negative result is also worth reporting. The first HistGradientBoosting configuration selected by CV (max_leaf_nodes=63, max_iter=300, learning_rate=0.2, class_weight=balanced; CV F1-macro 0.8421) diverged when refitted on the full training split: test F1-macro 0.3827, accuracy 0.4717. With balanced weights a Cottonwood/Willow cell weighs ≈100× more than a Lodgepole Pine cell, so its gradients are ≈100× larger, and a step of 0.2 overshoots. A pilot LightGBM run had shown the same behaviour. The search space was restricted to learning_rate ≤ 0.1 with optional L2 regularisation, and the refit converged (F1-macro 0.9546). Lesson: a configuration tuned on a subsample must be checked after refitting on the full data.

7.2 Learning curve

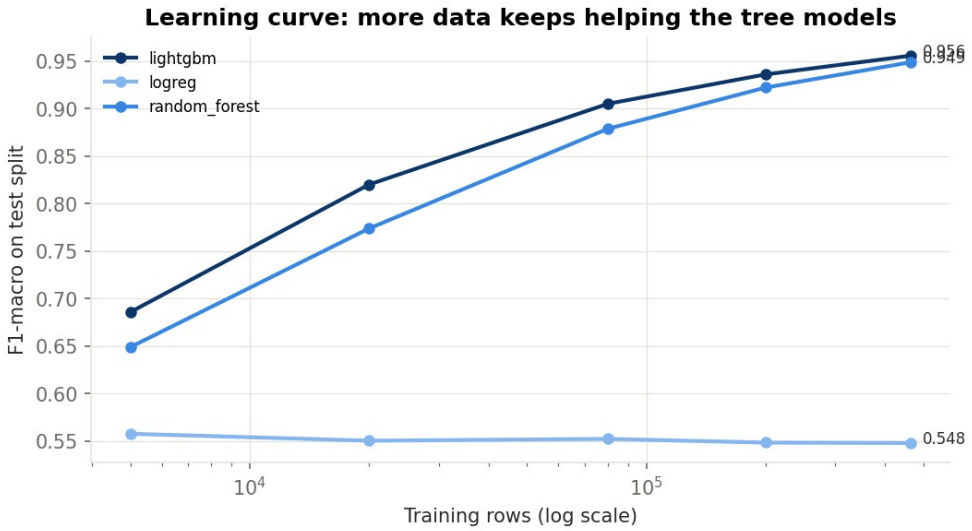


Figure 8. F1-macro vs training-set size (tuned parameters, same test split).

Logistic Regression does not improve with more data (F1-macro 0.557 at 5k rows, 0.548 at 465k): its error is bias, not variance. Random Forest and LightGBM keep improving up to the full training set (LightGBM: 0.686 → 0.936 → 0.956). This justifies processing the full data set instead of a sample, and is consistent with the original study, which trained on 11,340 cells and reached 70% accuracy with a neural network.

8. Interpretation and error analysis

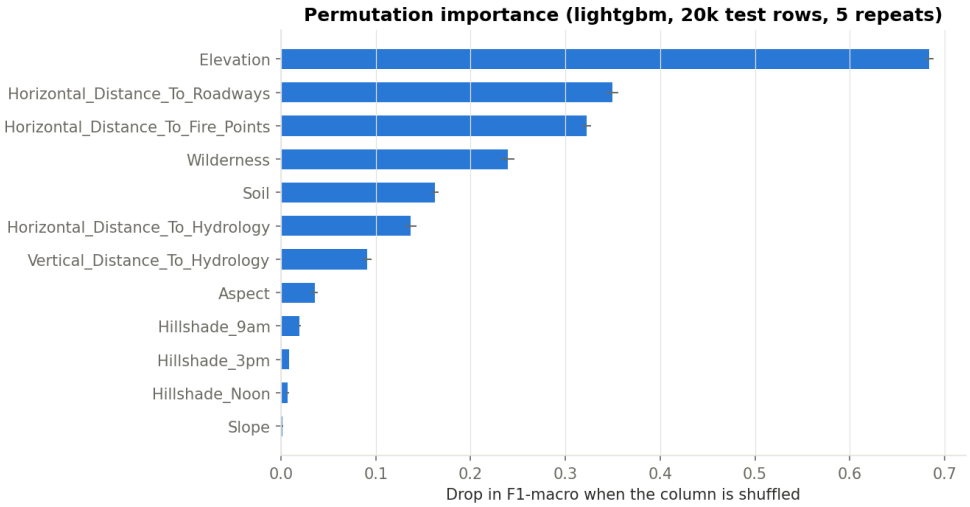


Figure 9. Permutation importance of the input columns for LightGBM.

Permutation importance was computed on 20,000 stratified test rows with 5 repeats, as the drop in F1-macro when one input column is shuffled (this also breaks every engineered feature built from it). Elevation dominates (0.684), followed by the distances to roads (0.349) and fire points (0.322), the wilderness area (0.239) and the soil type (0.162). Slope and the hillshade indices contribute almost nothing once the other features are known.

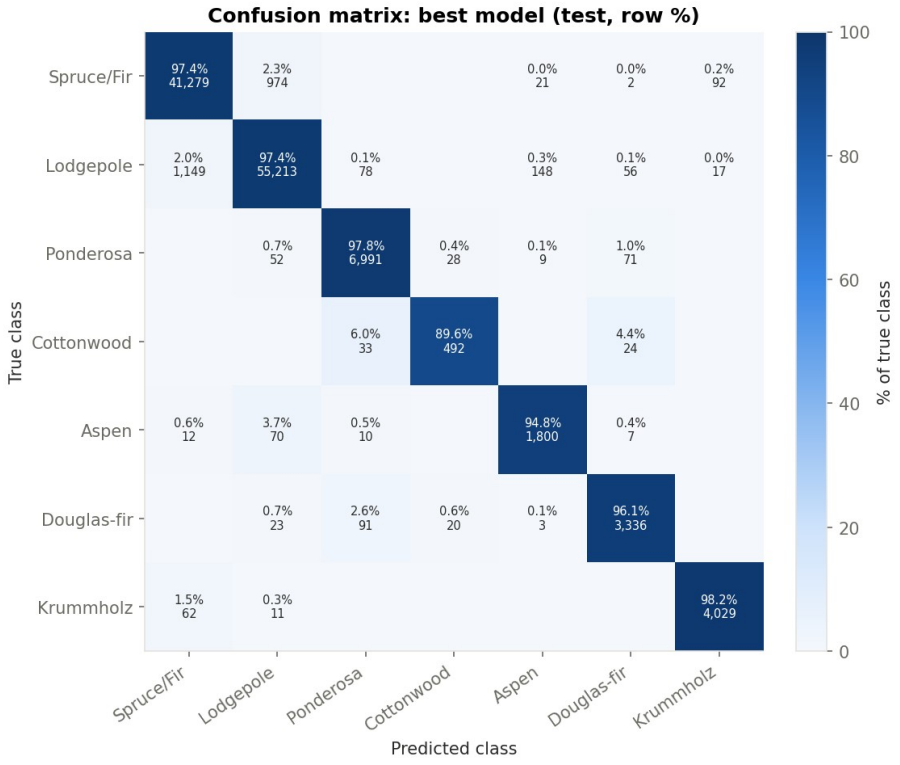


Figure 10. Confusion matrix of LightGBM (test split, row %).

The model makes 3,063 errors on 116,203 test cells. 2,123 of them (69%) are Spruce/Fir ↔ Lodgepole Pine confusions (SQL Q11), and 72% of all errors lie in the 2,750–3,250 m band where both species are common.

Low-elevation classes are confused among themselves (Douglas-fir ↔ Ponderosa Pine, Cottonwood/Willow → Ponderosa Pine), Krummholz only with Spruce/Fir at the tree line. The errors are therefore ecologically plausible: they occur between neighbouring vegetation belts, not at random.

The predicted probability is informative: 1.7% of test cells get a top probability below 0.6, and their error rate is 45%, against 1.9% for the rest. In practice such cells could be sent for field verification. Error rates differ little between wilderness areas (1.7–2.9%).

8.1 Group validation: a wilderness area the model has never seen

The random split puts neighbouring 30 m cells into both training and test data, and the data set has no coordinates for a spatially blocked split. The wilderness area is the only available spatial group, so we ran a leave-one-area-out check (src/robustness.py): train on three areas with the tuned parameters, test on all cells of the fourth. The variant 'LightGBM without area' sets the Wilderness column to a constant, because the code of the held-out area never occurs in training.

Table 10. F1-macro when the whole area is held out vs the random split (same area's cells of the 116k test split)

Held-out area	Test cells	LightGBM, random split	LightGBM, area held out	LightGBM w/o area, held out	RF, held out	Baseline, held out
Rawah	260,796	0.965	0.347	0.353	0.375	0.180
Neota	29,884	0.984	0.315	0.448	0.290	0.154
Comanche Peak	253,364	0.959	0.244	0.337	0.286	0.110
Cache la Poudre	36,968	0.956	0.253	0.271	0.265	0.038

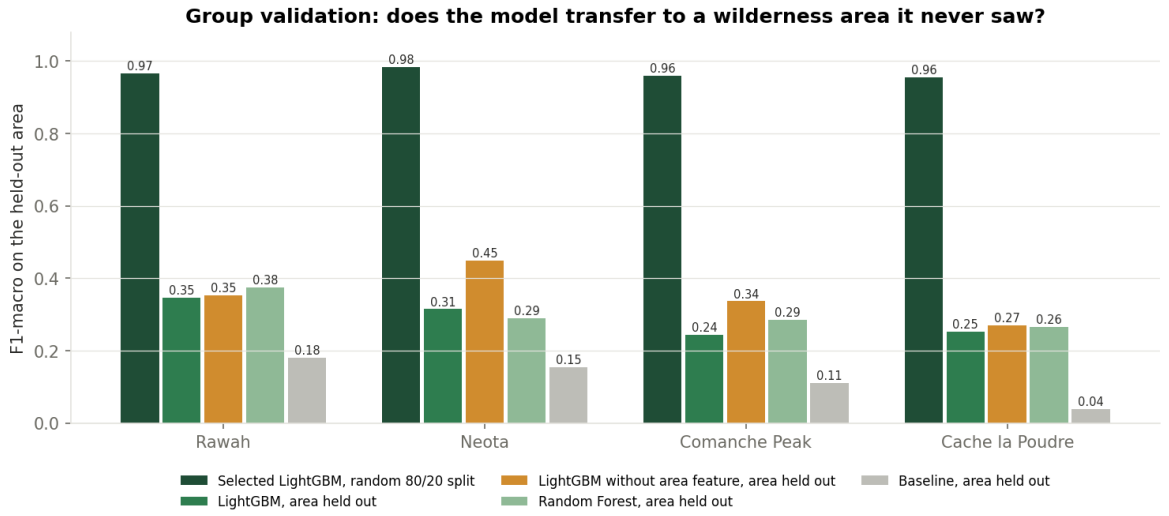


Figure 11. Group validation by wilderness area.

The result is a large and honest drop. On the random split the selected model reaches F1-macro 0.96–0.98 in every area; on an unseen area it falls to 0.24–0.35 (accuracy 0.42–0.57), still above the baseline (0.04–0.18). Part of the drop comes from the unknown area code: with the area feature removed, F1-macro improves by +0.01 to +0.13 (to 0.27–0.45). Most of the gap remains, however, so the main cause is a real shift: the relation between terrain, soil and species differs between areas (the same elevation belt carries a different species mix, and many soil types occur in one area only). Random Forest transfers no better than LightGBM. A practical consequence is that a model meant for new areas should not use the area code as a feature.

Cache la Poudre is the hardest case: Cottonwood/Willow occurs only there, so a model trained on the other areas cannot predict it at all.

Conclusion for the use of the model: the reported 0.956 F1-macro is valid for new cells inside the four mapped areas (the situation of the original study, filling gaps in a partly surveyed forest). Mapping a new area requires labelled cells from that area; the random-split score must not be quoted for it.

9. Discussion: which approach and why

- **Recommended model: LightGBM with balanced class weights.** Highest F1-macro, accuracy within 0.001 of the best, moderate size (≈91 MB in memory, 40 MB compressed on disk) and training time. Its weakness is prediction speed (≈0.4 s per 1,000 rows on 4 threads), irrelevant for an interactive tool but worth noting for map-scale inference.
- **Random Forest** is the choice when training and prediction time matter more than the last 0.7 point of F1-macro and memory is not a constraint.
- **Logistic Regression** is not adequate for this problem: it is fast and tiny but structurally unable to model the interactions that define the rare classes.
- **Data engineering** choices are justified by measurements: compact dtypes (−86% memory), Parquet (10× smaller, 5× faster to load), SQL for persistent joinable storage and analysis of predictions.

10. Extensions beyond the mandatory tasks

- **Two gradient-boosting libraries** (HistGradientBoosting and LightGBM) plus a controlled class-weight experiment, instead of the minimum of two models.
- **Group validation** by wilderness area (section 8.1), which measures transfer to unseen terrain.
- **Threshold-free metrics** ROC-AUC and PR-AUC for every model (Table 9).
- **Learning curve** that justifies processing the full 581k rows.
- **Predictions stored in SQLite** next to the data, with SQL error analysis (Q10–Q12).
- **Peak-memory measurement** of training with a background psutil thread (sees C++ allocations of LightGBM).
- **Two interfaces** (console menu and Streamlit dashboard) and a Colab-ready defense notebook; 7 automated tests; pinned dependency versions.
- **A documented negative result** (the diverged HistGB configuration) kept in reports/diverged_runs.json.

11. Limitations

- Spatial transfer: neighbouring 30 m cells are similar and the data set has no coordinates, so the random split places neighbours in both train and test. The leave-one-area-out check (section 8.1) shows that the score does not carry over to an unseen area (F1-macro 0.24–0.45 depending on the area and variant); a model for a new region must be trained with labelled cells from it.
- Hyper-parameters were tuned on a 40k-row subsample with a small random search; a larger search on the full data could change the ranking of the three tree ensembles, which are within 0.7 point of each other.
- Timing and memory figures come from a shared laptop and are only comparable within this study.

- The study area is a single national forest in Colorado; the model is not expected to transfer to other regions without retraining.

12. Conclusions

- Forest cover type can be predicted from terrain and soil alone with F1-macro 0.956 and accuracy 0.974 on 116k held-out cells.
- Gradient boosting (LightGBM, HistGradientBoosting) and Random Forest are all strong and within 0.7 point of F1-macro; LightGBM with class weights is the best trade-off between quality, size and time (HistGB is a close alternative with a smaller model).
- The linear model and the baseline show that the problem is defined by feature interactions, not by any single variable.
- Elevation, distances to roads and fire points, wilderness area and soil type carry most of the signal; remaining errors are between adjacent vegetation belts.
- The score holds for new cells inside the mapped areas, not for an unseen area: leave-one-area-out F1-macro is 0.24–0.35, and removing the area feature recovers only part of the gap, so the terrain-species relation itself differs between areas.
- Memory-aware loading, Parquet and a normalised SQLite database make the full 581k-row pipeline reproducible on a laptop with one command (python main.py --all).

13. Contributions of team members

Member	Contribution
Amanbayev Marlen (AAI-2601M)	Project architecture and repository; data download and chunked loading, dtype optimisation and memory measurements, Parquet and CSV-vs-Parquet benchmark (data_loader.py); SQLite schema, ER diagram and the 12 SQL queries (database.py, queries.py, er_diagram.py); console menu and Streamlit dashboard (main.py, app.py, predict.py); README and tests.
Aktaev Erik (AAI-2601M)	Data quality audit and cleaning decisions (audit.py); exploratory analysis and all EDA figures including the interactive Plotly chart (eda.py); interpretation of SQL results; sections 2–5 of the report.
Gajiyev Rashad (AAI-2601M)	Feature engineering and preprocessing pipelines (features.py); model zoo, cross-validated tuning and resource measurement (models.py, pipeline.py); evaluation, permutation importance, learning curve and error analysis (evaluate.py); sections 6–9 of the report.
All members	Presentation, conclusions, code review; each member can explain every module.

References

1. Blackard, J. A., & Dean, D. J. (1999). Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. Computers and Electronics in Agriculture, 24(3), 131–151.
2. Blackard, J. (1998). Covertype [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C50K5N
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
4. Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems 30.
5. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Appendix: how to reproduce

git clone https://github.com/mara-werils/forest-cover-classification.git && cd forest-cover-classification

pip install -r requirements.txt

python main.py --all # full pipeline; python main.py for the menu; streamlit run app.py for the dashboard